

Sérialisation JSON entre JavaScript et PHP

Souvent, dans une application web, il est nécessaire d'échanger des données entre le client (navigateur) et le serveur. Le format JSON (JavaScript Object Notation) est couramment utilisé pour ce type d'échange en raison de sa simplicité et de sa compatibilité avec JavaScript. Il est facile à lire et à écrire, ce qui le rend idéal pour la communication entre le client et le serveur.

Pour passer dans le corps d'une requête HTTP des données structurées, on utilise souvent le format JSON qui sera sérialisé en une chaîne de caractères et ensuite décodé avec le langage de programmation utilisé côté serveur, comme PHP.

1. Convertir objet JSON en chaînes de caractères (Sérialisation)

Il est souvent nécessaire de convertir un objet JavaScript en chaîne de caractères JSON pour l'envoyer à un serveur via une requête HTTP. Pour cela, on utilise la méthode `JSON.stringify()`.

La méthode `JSON.stringify()` prend en paramètre un objet JavaScript et renvoie une chaîne de caractères JSON.

```
const objet = {
  id: 1,
  tache: "Faire les courses",
  date: "2021-10-15",
  terminee: false,
  utilisateur_id: 1,
};

const chaineJSON = JSON.stringify(objet);

const config = {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: chaineJSON,
};

fetch("https://api.example.com/taches", config)
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.error(error));
```

Envoyer des données JSON avec Fetch

Pour s'assurer que le serveur comprenne que les données envoyées sont au format JSON, il est important de définir l'en-tête `Content-Type` de la requête HTTP sur `application/json`.

Le serveur peut ensuite lire les données JSON envoyées dans le corps de la requête et les traiter correctement.

2. Récupérer les données JSON via PHP (Désérialisation)

Pour récupérer les données JSON envoyées par un client, vous pouvez utiliser la fonction `file_get_contents('php://input')` en PHP. Cette fonction lit les données brutes du corps de la requête HTTP.

Ensuite, pour les convertir en un tableau associatif, vous pouvez utiliser la fonction `json_decode()`.

```
<?php

// Récupérer les données JSON du corps de la requête
$json = file_get_contents('php://input');

// Décoder les données JSON en un tableau associatif
$data = json_decode($json, true);

// ... traiter les données ...

?>
```

3. Traiter l'information et retourner les données JSON du serveur PHP vers le navigateur (Sérialisation)

Pour retourner des données JSON depuis un serveur PHP, vous pouvez utiliser la fonction `json_encode()` pour convertir un tableau associatif en une chaîne de caractères JSON. Pour s'assurer que le client comprend que les données renvoyées sont au format JSON, vous pouvez définir l'en-tête `Content-Type` de la réponse sur `application/json`.

```
<?php
// Fouiller dans la base de données MySQL pour obtenir les données
$data = mysqli_query($conn, "SELECT * FROM taches");

// Convertir les données en un tableau associatif
$taches = mysqli_fetch_all($data, MYSQLI_ASSOC);

// Retourner les données au format JSON
header('Content-Type: application/json');
echo json_encode($taches); //Retourne les données au format JSON sous forme de
chaînes de caractères

?>
```

4. Décoder une chaîne de caractères JSON en objet JSON par le navigateur depuis le serveur PHP (Désérialisation)

Pour décoder une chaîne de caractères JSON en un objet JavaScript, on utilise la méthode `JSON.parse()`. C'est l'inverse de `JSON.stringify()`. La méthode `.json()` de l'objet `Response` renvoie une promesse qui résout avec le corps de la réponse en tant qu'objet JSON et fait automatiquement appel à `JSON.parse()`.

```
const chaineJSON = '{"id":1,"tache":"Faire les courses","date":"2021-10-15","terminee":false,"utilisateur_id":1}';  
const objet = JSON.parse(chaineJSON);  
console.log(objet);
```

```
fetch("https://api.example.com/taches")  
  .then((response) => response.json())  
  .then((data) => {  
    const objet = JSON.parse(data);  
    console.log(objet);  
  })  
  .catch((error) => console.error(error));
```